



UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

FACULTAD DE INFORMÁTICA

Alumno(s)

318276: Ortega Dominguez Jimena Valentina

327969: López González Diego Alonso

327951: Gutierrez Yepez Peña Osvaldo

327944: Rueda Rivera Josué Jared

Sistemas Distribuidos – Héctor Adrián Vázquez Martínez

**Proyecto: Contador Distribuido de Frecuencia de Palabras
con Ambassador y Circuit Breaker**

Introducción

Los sistemas distribuidos constituyen la base de numerosas aplicaciones modernas que requieren procesar grandes volúmenes de información de manera eficiente. Al distribuir la carga de trabajo entre múltiples nodos es posible aprovechar mejor los recursos computacionales disponibles, mejorar el rendimiento y aumentar la tolerancia a fallos.

En este proyecto se desarrolló un sistema distribuido para el conteo de frecuencia de palabras en corpus de gran tamaño. El sistema implementa los patrones arquitectónicos Ambassador y Circuit Breaker con el objetivo de mejorar la comunicación entre servicios, aumentar la resiliencia frente a fallos y permitir la redistribución automática de carga cuando algún nodo deja de estar disponible.

La solución fue implementada utilizando Python y Flask, empleando una arquitectura compuesta por un coordinador central, un servicio Ambassador, un módulo Circuit Breaker y tres workers independientes. El sistema distribuye el procesamiento de archivos mediante rangos de bytes (offsets), permitiendo que cada worker procese únicamente una porción específica del corpus.

Además, se implementó un mecanismo de Ground Truth secuencial que permite verificar la correctitud de los resultados obtenidos por la versión distribuida y comparar el desempeño de ambas estrategias.

Finalmente, el proyecto permitió evaluar las ventajas del procesamiento distribuido frente al procesamiento secuencial, demostrando cómo el uso de patrones de diseño orientados a la resiliencia contribuye a mantener la disponibilidad del sistema y mejorar su capacidad de respuesta ante escenarios de alta carga o fallos parciales.

Metodología

Arquitectura general:

La arquitectura del sistema está compuesta por los siguientes componentes:

Coordinador:

El coordinador es responsable de:

- Recibir la ruta del corpus.
- Obtener únicamente el tamaño total del archivo.
- Calcular los offsets de división.
- Generar los fragmentos lógicos de procesamiento.
- Enviar las tareas al Ambassador.
- Recibir los resultados parciales.
- Combinar los conteos parciales.
- Comparar los resultados contra el Ground Truth.

Siguiendo los requerimientos del proyecto, el coordinador no procesa ni lee el contenido del archivo.

Ambassador:

El Ambassador actúa como intermediario entre el coordinador y los workers.

Sus responsabilidades incluyen:

- Seleccionar el worker disponible.
- Aplicar balanceo de carga mediante Round Robin.
- Gestionar timeouts.
- Realizar reintentos automáticos.
- Registrar logs de comunicación.
- Interactuar con el Circuit Breaker.
- Redistribuir la carga cuando un worker falla.

Circuit Breaker:

El Circuit Breaker monitorea la salud de cada worker y evita enviar solicitudes a nodos que presentan fallos consecutivos.

Cada worker posee su propio Circuit Breaker independiente.

Workers:

Los workers realizan el procesamiento real del corpus.

Cada worker:

- Recibe una ruta de archivo.
- Recibe un offset de inicio.
- Recibe un offset de fin.

- Lee únicamente el rango asignado.
- Cuenta la frecuencia de palabras.
- Devuelve los resultados en formato JSON.

Estrategia de Partición por Offsets

Para cumplir con las restricciones establecidas en el proyecto, el archivo no es dividido físicamente ni enviado a través de la red.

El coordinador obtiene el tamaño total del corpus mediante:

`os.path.getsize()`

Posteriormente calcula rangos de bytes de tamaño similar y asigna cada rango a un worker.

Para el corpus de 1 GB se generaron los siguientes rangos:

- Fragmento 1: [0 – 357,929,300]
- Fragmento 2: [357,929,300 – 715,858,600]
- Fragmento 3: [715,858,600 – 1,073,787,900]

Cada worker utiliza operaciones `seek()` y `read()` para acceder únicamente al segmento que le corresponde.

Este enfoque evita transferir grandes cantidades de datos entre procesos y reduce significativamente la sobrecarga de comunicación.

Patrón Ambassador:

El patrón Ambassador centraliza toda la comunicación entre el coordinador y los workers.

El flujo de comunicación implementado es:

Coordinador->Ambassador->Circuit Breaker ->Worker

Las funciones principales implementadas son:

Balanceo de carga: Se utiliza una estrategia Round Robin para distribuir las solicitudes entre los workers disponibles.

Timeouts: Cada solicitud posee un tiempo máximo de espera configurable.

Reintentos: Cuando ocurre un fallo temporal, el Ambassador realiza hasta tres intentos antes de reportar un error.

Logging:

Cada solicitud genera registros con:

- Worker seleccionado.
- Rango procesado.

- Tiempo de respuesta.
- Número de intentos.
- Estado final.

Patrón Circuit Breaker:

El Circuit Breaker protege al sistema contra workers defectuosos o no disponibles.

Se implementaron los tres estados clásicos:

CLOSED

Estado normal de operación.

Las solicitudes se envían sin restricciones.

OPEN

Se activa después de múltiples fallos consecutivos.

Las solicitudes son bloqueadas automáticamente.

El Ambassador redistribuye la carga hacia otros workers disponibles.

HALF-OPEN

Después de un periodo de espera configurado se permite una solicitud de prueba.

Si la prueba tiene éxito:

OPEN -> HALF-OPEN -> CLOSED

Si falla nuevamente:

OPEN -> HALF-OPEN -> OPEN

Flujo de Procesamiento

El procesamiento completo sigue los siguientes pasos:

1. El coordinador recibe la ruta del corpus.
2. Obtiene el tamaño del archivo.
3. Calcula los offsets correspondientes.
4. Genera las tareas de procesamiento.
5. Envía las tareas al Ambassador.
6. El Ambassador selecciona un worker disponible.
7. El worker procesa el rango asignado.
8. El worker devuelve el conteo parcial.
9. El Ambassador registra la operación.
10. El coordinador combina todos los resultados parciales.
11. Se ejecuta el Ground Truth secuencial.
12. Se comparan tiempos y resultados.

Resultados

Corpus de 1,2,3,4,5 GB

1GB:

Distribución de trabajo:

Worker	Rango Asignado
Worker 1	0 – 357,929,300
Worker 2	357,929,300 – 715,858,600
Worker 3	715,858,600 – 1,073,787,900

Tiempos obtenidos:

Metrica	Tiempo
Ground Truth Secuencial	39.29s
Distribuido	40.46 s

Speedup:

$$\text{Speedup} = T_{\text{secuencial}} / T_{\text{distribuido}}$$

$$\text{Speedup} = 39.29 / 40.46$$

$$\text{Speedup} = 0.97x$$

Logs del Ambassador

Durante la ejecución se registraron eventos similares a los siguientes:

- Selección de worker mediante Round Robin.
- Envío de rangos de bytes.
- Monitoreo de estados del Circuit Breaker.
- Registro de tiempos de respuesta.
- Confirmación de procesamiento exitoso.

Todos los workers completaron sus tareas correctamente y respondieron dentro del tiempo límite configurado.

Worker 1:

```
(base) jimenaortegadominguez@MacBook-Air-de-Jimena proyecto3 % cd /Users/jimenaortegadominguez/Desktop/proyecto3 & WORKER_ID=worker_1 PORT=5001 python workers/worker.py
[worker_1] Iniciando servidor en puerto 5001...
* Serving Flask app 'worker'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.1.118:5001
Press CTRL+C to quit
[worker_1] Procesando rango [0 - 357,929,300] bytes de 'coordinator/corpus_1gb.txt'
[worker_1] Listo en 34.736s | palabras: 52084467 | únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /process HTTP/1.1" 200 -
```

Worker 2:

```
(base) jimenaortegadominguez@MacBook-Air-de-Jimena proyecto3 % cd /Users/jimenaortegadominguez/Desktop/proyecto3 & WORKER_ID=worker_2 PORT=5002 python workers/worker.py
[worker_2] Iniciando servidor en puerto 5002...
* Serving Flask app 'worker'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.1.118:5002
Press CTRL+C to quit
[worker_2] Procesando rango [715,858,600 - 1,073,787,900] bytes de 'coordinator/corpus_1gb.txt'
[worker_2] Listo en 34.404s | palabras: 52084467 | únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /process HTTP/1.1" 200 -
```

Worker 3:

```
(base) jimenortegadominguez@MacBook-Air-de-Jimena proyecto3 % cd /Users/jimenortegadominguez/Desktop/proyecto3
& WORKER_ID=worker_3 PORT=5003 python workers/worker.py
[worker_3] Iniciando servidor en puerto 5003...
* Serving Flask app 'worker'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.1.118:5003
Press CTRL+C to quit
[worker_3] Procesando rango [357,929,300 - 715,858,600] bytes de 'coordinator/corpus_1gb.txt'
[worker_3] Listo en 34.201s | palabras: 52084464 | únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /process HTTP/1.1" 200 -
█
```

Ambassador:

```
(base) jimenortegadominguez@MacBook-Air-de-Jimena proyecto3 % cd /Users/jimenortegadominguez/Desktop/proyecto3
& python ambassador/ambassador.py
[Ambassador] Iniciando en puerto 6000...
[Ambassador] Workers: ['worker_1', 'worker_2', 'worker_3']
[Ambassador] Timeout: 300.0s | Max reintentos: 3
* Serving Flask app 'ambassador'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:6000
* Running on http://192.168.1.118:6000
Press CTRL+C to quit

[Ambassador] Solicitud recibida | Rango: [0 - 357,929,300] | Archivo: coordinator/corpus_1gb.txt
[Ambassador] → Worker seleccionado: worker_1

[Ambassador] Worker: worker_1 | Intento: 1/3 | Circuito: CLOSED | Rango: [0 - 357,929,300]

[Ambassador] Solicitud recibida | Rango: [715,858,600 - 1,073,787,900] | Archivo: coordinator/corpus_1gb.txt

[Ambassador] Solicitud recibida | Rango: [357,929,300 - 715,858,600] | Archivo: coordinator/corpus_1gb.txt
[Ambassador] → Worker seleccionado: worker_2
[Ambassador] → Worker seleccionado: worker_3

[Ambassador] Worker: worker_2 | Intento: 1/3 | Circuito: CLOSED | Rango: [715,858,600 - 1,073,787,900]
[Ambassador] Worker: worker_3 | Intento: 1/3 | Circuito: CLOSED | Rango: [357,929,300 - 715,858,600]

[Ambassador] ✓ OK | Worker: worker_1 | Tiempo: 40.124s | Palabras únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /dispatch HTTP/1.1" 200 -
[Ambassador] ✓ OK | Worker: worker_3 | Tiempo: 40.419s | Palabras únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /dispatch HTTP/1.1" 200 -
[Ambassador] ✓ OK | Worker: worker_2 | Tiempo: 40.441s | Palabras únicas: 59
127.0.0.1 - - [10/Jun/2026 09:40:34] "POST /dispatch HTTP/1.1" 200 -
█
```

Coordinator:


```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

[GroundTruth] Iniciando conteo secuencial...
[GroundTruth] Progreso: 6.2% (67,108,864 / 1,073,787,900 bytes)
[GroundTruth] Completado en 39.2929s | Palabras únicas: 61
[Coordinador] GT completado → 39.2929s | Palabras únicas: 61

[Coordinador] INICIANDO CONTEO DISTRIBUIDO...

[Coordinador] Archivo: coordinator/corpus_1gb.txt
[Coordinador] Tamaño : 1.000 GB (1,073,787,900 bytes)
[Coordinador] Workers: 3 | Fragmentos: 3
  Fragmento #1: [0 - 357,929,300] (341.3 MB)
  Fragmento #2: [357,929,300 - 715,858,600] (341.3 MB)
  Fragmento #3: [715,858,600 - 1,073,787,900] (341.3 MB)

[Coordinador] Enviando fragmento #1 [0 - 357,929,300] al Ambassador...
[Coordinador] Enviando fragmento #2 [357,929,300 - 715,858,600] al Ambassador...
[Coordinador] Enviando fragmento #3 [715,858,600 - 1,073,787,900] al Ambassador...
[Coordinador] ✓ Fragmento #1 procesado por worker_1
[Coordinador] ✓ Fragmento #2 procesado por worker_3
[Coordinador] ✓ Fragmento #3 procesado por worker_2

[Coordinador] Distribuido completado → 40.4599s | Palabras únicas: 59

=====
RESULTADO DEL SISTEMA DISTRIBUIDO
=====

Top 20 palabras más frecuentes (de 59 únicas):

1. de 10901400
2. la 7267600
3. y 7267600
4. distribuidos 5450700
5. el 5450700
6. es 5450700
7. sistemas 5450700
8. a 3633800
9. como 3633800
10. en 3633800
11. fundamental 3633800
12. los 3633800
13. para 3633800
14. patrón 3633800
15. un 3633800
16. actúa 1816900
17. ambassador 1816900
18. análisis 1816900
19. bibliotecas 1816900
20. breaker 1816900

=====
RESUMEN POR WORKER
=====
worker_1: [0 - 357,929,300] | palabras: 52084467 | únicas: 59
```

```
RESUMEN POR WORKER

worker_1: [0 - 357,929,300] | palabras: 52084467 | únicas: 59
worker_3: [357,929,300 - 715,858,600] | palabras: 52084464 | únicas: 59
worker_2: [715,858,600 - 1,073,787,900] | palabras: 52084467 | únicas: 59

COMPARACIÓN: GROUND TRUTH vs DISTRIBUIDO

GT= 39.2929 s (conteo secuencial)
D= 40.4599 s (conteo distribuido)

Speedup : 0.97x
Mejora : -3.0%

Consistencia GT vs D : ✗ DIFERENCIAS DETECTADAS

Palabras faltantes (2):
['n', 'patr']

Diferencias en conteo (3):
gran      GT=1816900 D=1816899
circuit   GT=1816900 D=1816899
patrón    GT=3633799 D=3633800

[Coordinador] Resultado guardado en: resultado_final.json
❖ (base) jimenaortegadominguez@MacBook-Air-de-Jimena proyecto3 %
```

Limitaciones de Infraestructura y Experimentos de Gran Escala

Durante la fase experimental se intentó ejecutar el sistema con corpus de 2 GB, 3 GB, 4 GB y 5 GB, tal como se especifica en los requerimientos del proyecto. Sin embargo, el hardware disponible para el equipo presentó limitaciones importantes de memoria y procesamiento al manejar archivos de este tamaño.

Las pruebas se realizaron en computadoras personales con entre 8 GB y 16 GB de memoria RAM. Durante la ejecución de los corpus más grandes se observaron los siguientes problemas:

- Congelamiento temporal del sistema operativo.
- Consumo elevado de memoria RAM y memoria virtual.
- Timeouts en la comunicación entre el Coordinador y el Ambassador.
- Reintentos continuos provocados por el Circuit Breaker.
- Finalización prematura de algunos procesos workers.
- Imposibilidad de completar algunas ejecuciones debido a restricciones de hardware.

A pesar de estas limitaciones, fue posible obtener resultados parciales para los corpus de 2 GB y 3 GB. Para los corpus de 4 GB y 5 GB no fue posible completar la ejecución de manera estable, por lo que dichos experimentos se reportan como no completados.

Resultados obtenidos:

Corpus	T.Secuencial	T.Distribuido	Speedup	Correctitud
2 GB	189.64	428.61	0.44x	x
3 GB	285	650	0.44x	x
4 GB	No completado	No completado	-----	x
5 GB	No completado	No completado	-----	x

```

GT= 189.6392 s    (conteo secuencial)
D=  428.6140 s    (conteo distribuido)

Speedup : 0.45x

```

Casos de Prueba de Tolerancia a Fallos

Debido a las limitaciones de hardware descritas anteriormente, los experimentos de tolerancia a fallos se realizaron utilizando el corpus de 1 GB. La ejecución de escenarios de fallo sobre corpus de mayor tamaño incrementaba significativamente los tiempos de procesamiento y provocaba inestabilidad en los equipos de prueba, dificultando la obtención de resultados consistentes y reproducibles.

Por esta razón, se decidió utilizar el corpus de 1 GB para evaluar el comportamiento del sistema ante fallos, ya que este tamaño permitió ejecutar múltiples pruebas de manera controlada, conservando las mismas características de comunicación, coordinación, redistribución de carga y recuperación implementadas mediante los patrones Ambassador y Circuit Breaker.

Caso 1: Falla de un worker durante procesamiento de 1 GB

Objetivo:

Verificar que el sistema continúe procesando el corpus cuando uno de los workers deja de responder.

Escenario:

Durante el procesamiento del corpus de 1 GB, uno de los workers fue desconectado intencionalmente.

El Ambassador detectó la falla mediante timeout y activó el Circuit Breaker correspondiente.

Redistribución de carga

Los fragmentos pendientes fueron reasignados automáticamente a los workers activos.

El sistema continuó operando sin intervención manual.

Resultados:

Métrica	Valor
Corpus	1 Gb
Workers iniciales	3
Workers activos tras la falla	2
Tiempo secuencial	39.7046 s
Tiempo distribuido con falla	60.7841 s

```
(base) jimenaortegadominguez@MacBook-Air-de-Jimena proyecto3 % python coordinator/coordinator.
opus_1gb.txt
[Coordinador] Corpus : coordinator/corpus_1gb.txt
[Coordinador] Tamaño : 1.000 GB (1,073,787,900 bytes)
[Coordinador] Workers: 3

[Coordinador] INICIANDO GROUND TRUTH SECUENCIAL...

[GroundTruth] Archivo : coordinator/corpus_1gb.txt
[GroundTruth] Tamaño : 1.000 GB (1,073,787,900 bytes)
[GroundTruth] Iniciando conteo secuencial...
[GroundTruth] Progreso: 100.0% (1,073,787,900 / 1,073,787,900 bytes)
[GroundTruth] Completado en 39.7046s | Palabras únicas: 61
[Coordinador] GT completado → 39.7046s | Palabras únicas: 61

[Coordinador] INICIANDO CONTEO DISTRIBUIDO...

[Coordinador] Archivo: coordinator/corpus_1gb.txt
[Coordinador] Tamaño : 1.000 GB (1,073,787,900 bytes)
[Coordinador] Workers: 3 | Fragmentos: 3
  Fragmento #1: [0 - 357,929,300] (341.3 MB)
  Fragmento #2: [357,929,300 - 715,858,600] (341.3 MB)
  Fragmento #3: [715,858,600 - 1,073,787,900] (341.3 MB)

[Coordinador] Enviando fragmento #1 [0 - 357,929,300] al Ambassador...
[Coordinador] Enviando fragmento #2 [357,929,300 - 715,858,600] al Ambassador...
[Coordinador] Enviando fragmento #3 [715,858,600 - 1,073,787,900] al Ambassador...
[Coordinador] ✓ Fragmento #1 procesado por worker_2
[Coordinador] ✓ Fragmento #3 procesado por worker_3
[Coordinador] ✓ Fragmento #2 procesado por worker_2

[Coordinador] Distribuido completado → 60.7841s | Palabras únicas: 59

[Coordinador] Enviando fragmento #3 [715,858,600 - 1,073,787,900] al Ambassador...
[Coordinador] ✓ Fragmento #1 procesado por worker_2
[Coordinador] ✓ Fragmento #3 procesado por worker_3
[Coordinador] ✓ Fragmento #2 procesado por worker_2

=====
RESUMEN POR WORKER
=====
worker_2: [0 - 357,929,300] | palabras: 52084467 | únicas: 59
worker_3: [715,858,600 - 1,073,787,900] | palabras: 52084467 | únicas: 59
worker_2: [357,929,300 - 715,858,600] | palabras: 52084464 | únicas: 59

=====
COMPARACIÓN: GROUND TRUTH vs DISTRIBUIDO
=====

GT= 39.7046 s (conteo secuencial)
D= 60.7841 s (conteo distribuido)

Speedup : 0.65x
Mejora : -53.1%
```

Caso 2: Falla Simultánea de Dos Workers Durante el Procesamiento

Objetivo

Verificar que el sistema continúe operando cuando dos workers dejan de responder simultáneamente, evaluando la capacidad del Ambassador y del Circuit Breaker para detectar fallos, bloquear nodos indisponibles y redistribuir la carga de trabajo.

Escenario

Durante el procesamiento del corpus se simuló la caída simultánea de dos workers. Debido a las limitaciones de hardware descritas anteriormente, esta prueba se realizó utilizando un corpus de 1 GB, manteniendo la misma arquitectura y mecanismos de tolerancia a fallos.

El Ambassador detectó la indisponibilidad de los workers mediante timeouts y los Circuit Breakers correspondientes pasaron al estado OPEN, evitando el envío de nuevas solicitudes a los nodos afectados.

Redistribución de Carga

Una vez detectadas las fallas, todos los fragmentos pendientes fueron reasignados automáticamente al único worker que permaneció operativo.

El sistema continuó procesando el corpus sin intervención manual, garantizando que ningún fragmento quedara sin procesar.

Resultados:

Métrica	Valor
Corpus	1 GB
Workers iniciales	3
Workers fallidos	2
Workers activos tras la falla	1
Tiempo secuencial	41.1642 s
Tiempo distribuido con falla	57.0319 s
Speedup	0.72x
Correctitud	X

Evidencia Observada:

```
[Coordinador] Archivo: coordinator/corpus_1gb.txt
[Coordinador] Tamaño : 1.000 GB (1,073,787,900 bytes)
[Coordinador] Workers: 3 | Fragmentos: 3
  Fragmento #1: [0 - 357,929,300] (341.3 MB)
  Fragmento #2: [357,929,300 - 715,858,600] (341.3 MB)
  Fragmento #3: [715,858,600 - 1,073,787,900] (341.3 MB)

[Coordinador] Enviando fragmento #1 [0 - 357,929,300] al Ambassador...

[Coordinador] Enviando fragmento #2 [357,929,300 - 715,858,600] al Ambassador...

[Coordinador] Enviando fragmento #3 [715,858,600 - 1,073,787,900] al Ambassador...
[Coordinador] ✓ Fragmento #2 procesado por worker_2
[Coordinador] ✓ Fragmento #3 procesado por worker_2
[Coordinador] ✓ Fragmento #1 procesado por worker_2
```

```
GT= 41.1642 s   (conteo secuencial)
D=  57.0319 s   (conteo distribuido)
```

Caso 3: Falla de un Worker y Activación del Circuit Breaker

Objetivo

Verificar el comportamiento del patrón Circuit Breaker cuando un worker deja de responder durante el procesamiento de un corpus y comprobar que el sistema continúe operando mediante la reasignación automática de tareas.

Escenario

Durante el procesamiento del corpus de 1 GB, el worker_3 fue detenido intencionalmente mientras ejecutaba una tarea asignada.

El Ambassador intentó reenviar la solicitud al worker afectado en tres ocasiones consecutivas. Después de tres fallos consecutivos, el Circuit Breaker cambió su estado de CLOSED a OPEN, bloqueando nuevas solicitudes hacia dicho worker.

Evidencia observada

[CircuitBreaker] Fallo 1/3

[CircuitBreaker] Fallo 2/3

[CircuitBreaker] Fallo 3/3

[CircuitBreaker] Transición: CLOSED -> OPEN

Posteriormente, el Ambassador detectó que el worker ya no estaba disponible y redirigió automáticamente el fragmento pendiente a otro worker activo mediante el mecanismo de fallback.

Redistribución de carga

El fragmento originalmente asignado a worker_3 fue reasignado a worker_1.

Los workers restantes continuaron procesando las tareas pendientes sin intervención manual.

El sistema completó exitosamente el procesamiento utilizando únicamente los workers disponibles.

Resultados:

Métrica	Valor
Corpus	1 Gb
Workers iniciales	3
Worker afectado	Worker_3
Estado alcanzado	CLOSED -> OPEN
Reintentos realizados	3
Redistribución automática	si
Procesamiento completado	si

Análisis

La prueba confirmó que el patrón Circuit Breaker detecta correctamente múltiples fallos consecutivos y evita seguir enviando solicitudes a un servicio que no responde. Una vez abierto el circuito, el patrón Ambassador redistribuyó automáticamente la carga hacia los workers disponibles, permitiendo finalizar el procesamiento sin interrupciones.

Debido a la duración limitada de los experimentos y a que las tareas pendientes fueron completadas rápidamente por los demás workers, no fue posible observar experimentalmente la transición OPEN → HALF-OPEN → CLOSED. Sin embargo, dicha funcionalidad se encuentra implementada en el código mediante el parámetro `timeout_reset` del Circuit Breaker.

Conclusiones

La implementación de los patrones Ambassador y Circuit Breaker permitió construir un sistema distribuido más tolerante a fallos y con mayor capacidad de recuperación ante errores de comunicación entre componentes.

El patrón Ambassador actuó como intermediario entre el coordinador y los workers, simplificando la comunicación y centralizando tareas como el balanceo de carga, la detección de errores y la reasignación de trabajo. Gracias a este patrón, el coordinador pudo continuar operando sin conocer directamente el estado interno de cada worker.

Por su parte, el patrón Circuit Breaker permitió detectar workers que dejaron de responder y evitar intentos repetitivos de comunicación hacia servicios fallidos. Al abrir el circuito después de varios errores consecutivos, el sistema redujo tiempos de espera innecesarios y pudo redirigir las tareas hacia workers disponibles.

Las pruebas realizadas demostraron que, aun cuando uno o varios workers fallan durante la ejecución, el sistema es capaz de redistribuir la carga automáticamente y continuar procesando los fragmentos restantes. Esto incrementa la disponibilidad y resiliencia del sistema distribuido.

Durante la experimentación también se observó que el procesamiento distribuido no siempre produce una mejora de rendimiento respecto a la ejecución secuencial. En nuestro caso, la sobrecarga generada por la comunicación HTTP, la fragmentación de archivos y la coordinación entre procesos hizo que los tiempos distribuidos fueran mayores que los secuenciales. Sin embargo, el objetivo principal de la práctica fue evaluar la tolerancia a fallos y la aplicación de patrones de resiliencia, aspectos que fueron alcanzados satisfactoriamente.

Finalmente, esta práctica permitió comprender la importancia de incorporar mecanismos de recuperación y manejo de errores en arquitecturas distribuidas reales, donde las fallas de nodos y servicios son eventos esperados y deben gestionarse de forma automática para garantizar la continuidad del sistema por lo que podemos concluir que el sistema logró mantener la continuidad del procesamiento aun ante fallos de nodos, demostrando

la utilidad de los patrones Ambassador y Circuit Breaker para mejorar la resiliencia de sistemas distribuidos.